

SmartFetch: High-Level Report

Adaptive Web Data Extraction Web Application

Youssef Mabrouk Adam Hajri Jaouher Chouchane

Mohammed Sanad

Information Assurance and Security

May 2026

Abstract

SmartFetch is a client-server web application for adaptive web data extraction from one user-provided webpage at a time. The system accepts a public URL, retrieves and analyzes the HTML, suggests filters based on detected content, allows the user to select the desired data types, previews cleaned results, and exports the output as JSON or CSV. This report summarizes the purpose, scope, architecture, data flow, implementation, security controls, testing approach, and limitations of the final SmartFetch web application.

Contents

1	Introduction	3
2	Project Objective	3
3	Scope	3
4	Web Scraping Concepts	3
5	Final Application Overview	4
6	Technology Stack	4
7	High-Level Architecture	4
8	System Flow	4
9	API Design	4
9.1	Analyze Endpoint	4
9.2	Extract Endpoint	5
10	Adaptive Filters	5
11	Separate and Grouped Extraction Modes	5
12	Data Cleaning	6
13	Export Design	6
14	Security and Ethical Considerations	6
15	Testing Strategy	7

16 Limitations	7
17 Future Improvements	7
18 Conclusion	7

1 Introduction

Manual collection of web data is slow, repetitive, and prone to errors. Websites also present information using different layouts and HTML structures, which means a scraper designed for one website can fail when the page structure changes. SmartFetch addresses this problem by analyzing the current webpage first and then generating extraction filters dynamically.

The project is intentionally limited to a single webpage supplied by the user. It is not a crawler and does not automatically follow links. This design keeps the application easier to control, safer to test, and realistic for the project scope.

2 Project Objective

The main objective of SmartFetch is to provide a controlled and adaptive data extraction workflow:

1. Accept one public webpage URL from the user.
2. Validate the URL and retrieve the page HTML.
3. Analyze the HTML structure to detect available data types.
4. Suggest relevant filters automatically.
5. Extract only the data selected by the user.
6. Clean the extracted output by trimming whitespace, removing empty values, and removing duplicates.
7. Preview the results in the browser.
8. Export the final result as JSON or CSV.

3 Scope

SmartFetch focuses on adaptive extraction from one webpage at a time. It supports static HTML retrieval and parsing through the backend. The current system does not execute JavaScript on target websites, does not crawl multiple pages, and does not log into websites.

Included in Scope	Outside Current Scope
Single public webpage analysis	Full-site crawling
HTML parsing with Cheerio	JavaScript-rendered page execution
Adaptive filter suggestions	Login or authenticated scraping
Preview and export as JSON/CSV	Scheduled or background scraping
Basic URL and SSRF protection	Proxy rotation or anti-bot bypassing

Table 1: SmartFetch scope boundaries.

4 Web Scraping Concepts

A web scraper generally sends an HTTP request to a target website, receives HTML as the response, parses the HTML structure, extracts selected data, cleans the result, and stores or exports it in a structured format. SmartFetch uses these same concepts but avoids crawler behavior. Instead of discovering many pages, it processes only the URL entered by the user.

5 Final Application Overview

The final SmartFetch application provides a modern web interface with four main sections:

- **URL Input:** the user enters the target webpage URL and starts analysis.
- **Suggested Filters:** the application displays available filters based on detected elements.
- **Preview:** extracted data is shown in a readable table or, for grouped mode, an optional card view.
- **Export:** the user downloads the latest extraction as JSON or CSV.

6 Technology Stack

Technology	Role in SmartFetch
HTML	Defines the frontend page structure.
CSS	Provides the dark navy visual design, responsive layout, panels, buttons, and preview styling.
JavaScript	Handles frontend interactivity, API calls, filter selection, preview rendering, recent URL history, and downloads.
Node.js	Runs the backend server.
Express.js	Defines API routes and serves the static frontend.
Axios	Retrieves HTML from the target webpage with timeout and redirect handling.
Cheerio	Parses HTML on the backend and extracts elements using CSS selectors.
JSON	Used for API communication and export.
CSV	Used for spreadsheet-friendly export.

Table 2: Main implementation technologies.

7 High-Level Architecture

SmartFetch follows a client-server architecture. The browser handles user interaction, while the backend performs validation, HTML retrieval, analysis, extraction, cleaning, and export preparation.

8 System Flow

The complete SmartFetch workflow is divided into analysis and extraction phases.

9 API Design

9.1 Analyze Endpoint

The POST `/api/analyze` endpoint receives a URL, validates it, retrieves the webpage, parses the HTML, and returns the detected filters. The frontend uses this response to display selectable filter cards.

9.2 Extract Endpoint

The POST `/api/extract` endpoint receives a URL, selected filters, and the grouped-mode option. It extracts the selected data, cleans the result, computes a summary, and returns structured sections plus a CSV string.

Endpoint	Input	Output
<code>/api/analyze</code>	URL	Final URL, page title, suggested filters, counts
<code>/api/extract</code>	URL, filters, grouped mode	Extracted sections, summary, export settings, CSV data

Table 3: SmartFetch API endpoints.

10 Adaptive Filters

SmartFetch detects the following filters from the target HTML:

Filter	Detection Source	Extracted Data
Headings	h1, h2, h3	Level and text
Paragraphs	p tags	Readable text content
Links	a[href]	Link text and resolved URL
Images	img	Image source and alt text
Tables	table, tr, th, td, and selected table-like rows	Row values and structured columns when possible
Lists	ul, ol, li	List items grouped by list type
Metadata	title, meta tags	Title, description, keywords, Open Graph, and social meta-data
Prices	Price-like text patterns	Values using symbols or formats such as \$, EUR, TND, and numeric price patterns
Dates	time, datetime, and date-like text	Date values from semantic tags and page text

Table 4: Detected SmartFetch filters.

11 Separate and Grouped Extraction Modes

SmartFetch supports two extraction modes:

- **Separate mode:** each selected filter is extracted independently. This is best for tables and when the user wants a simple list of one data type.
- **Grouped mode:** the backend attempts to preserve relationships between elements that appear inside the same card, article, product block, search result, or section. This is useful for pages where a heading, image, price, date, paragraph, and link belong to the same item.

Grouped extraction was added to better match real web layouts such as news cards, product listings, and repeated search results. The preview supports both table view and card view only when grouped mode is active.

12 Data Cleaning

Raw web data can include empty strings, repeated values, extra spaces, navigation links, footer links, or duplicated elements. SmartFetch applies cleaning before preview and export:

- trims whitespace and normalizes repeated spaces;
- removes empty rows and empty values;
- removes duplicate rows;
- resolves relative URLs into absolute URLs;
- reduces noisy navigation, footer, account, cookie, and repeated site-wide links;
- deduplicates grouped links by destination URL;
- keeps image previews as thumbnails in the browser while exporting image URLs in JSON and CSV.

13 Export Design

SmartFetch provides two export formats:

- **JSON export:** preserves the full response structure, including sections, selected filters, grouped mode, source URL, extraction duration, and extracted rows.
- **CSV export:** creates spreadsheet-friendly rows. The export begins with a small settings block containing the source URL, grouped-mode status, selected filters, total rows, and extraction duration.

For complex table data, the backend attempts to keep meaningful columns, such as month or change fields in table-like layouts. For images, exports store URLs rather than embedding binary image data.

14 Security and Ethical Considerations

SmartFetch is designed for public webpage data only. It should not be used to collect private, sensitive, or personally identifiable information. The implementation includes several basic protections:

- rejects invalid URLs;
- allows only `http` and `https` protocols;
- blocks localhost and private IP address ranges where possible;
- validates redirects before following them;
- limits request duration with a timeout;
- limits HTML response size;
- processes one URL at a time instead of crawling many pages.

These controls reduce risk but do not replace responsible use. Users should respect website terms, robots policies, and request frequency limits.

15 Testing Strategy

The project includes automated backend tests using Node’s built-in test runner. The tests cover:

- whitespace normalization and duplicate removal;
- extraction of headings, links, images, tables, prices, dates, lists, and metadata;
- grouped extraction behavior;
- splitting large grouped wrappers into smaller cards;
- duplicate link removal in grouped mode;
- page-level date handling;
- table-like div extraction, including Steam-style hardware rows;
- CSV export with settings;
- private IP blocking and URL resolution.

At the time of this report, the test suite passes with 11 successful tests and 0 failures.

16 Limitations

SmartFetch is functional and ready for demonstration, but it still has realistic limitations:

- it does not execute JavaScript on target websites, so content rendered after page load may not appear in the fetched HTML;
- grouped extraction uses heuristics, so very unusual layouts may produce imperfect grouping;
- it does not handle login-protected pages;
- it does not include custom user-defined CSS selectors;
- it is intentionally not a full crawler.

17 Future Improvements

Possible future improvements include:

- optional browser automation for JavaScript-heavy websites;
- saved custom filter presets;
- more advanced repeated-block detection;
- controlled multi-page crawling with strict limits;
- richer export templates for specific domains such as products, news, or research data;
- a small in-app explanation panel for grouped mode and each filter.

18 Conclusion

SmartFetch successfully implements the high-level project design as a working web application. It combines a clean frontend interface with a backend analysis and extraction pipeline. The adaptive filter approach makes it more flexible than a fixed scraper, while the single-page scope

and URL protections keep it controlled and suitable for academic demonstration. The final application supports analysis, extraction, preview, grouped mode, data cleaning, and JSON/CSV export, fulfilling the main objectives described in the project documents.

ADAPTIVE WEB DATA EXTRACTION

SmartFetch

Analyze one webpage, choose suggested filters, preview clean data, then export JSON or CSV.

Ready

1 URL Input

Enter a public webpage URL. SmartFetch supports one page at a time.

Webpage URL

https://example.com/products

Analyze

2 Suggested Filters

Detected filters appear after analysis.

☐ Grouped extraction mode

Best for cards, articles, products, and search results. For tables, keep this off so rows and columns stay easier to read.

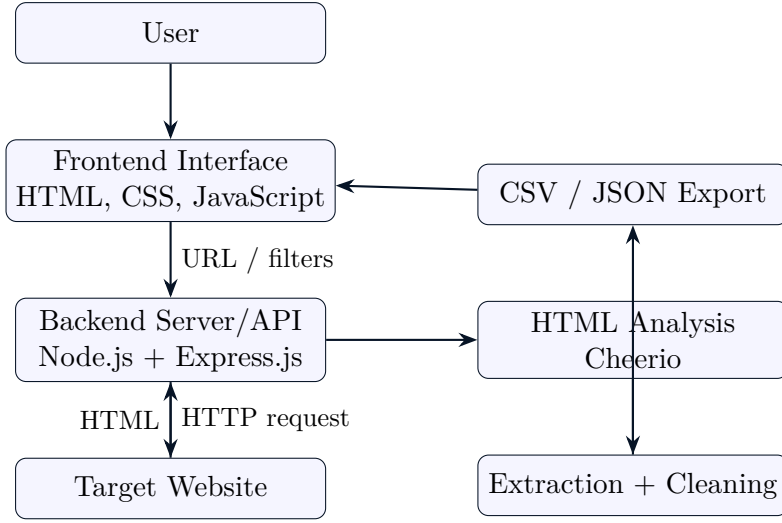


Figure 2: SmartFetch high-level architecture.

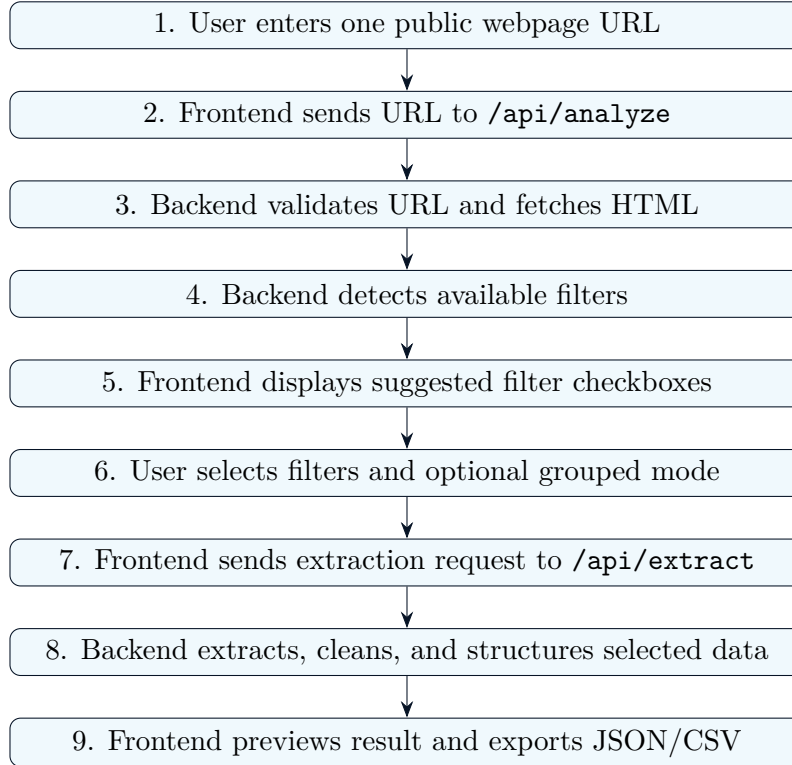


Figure 3: End-to-end SmartFetch workflow.